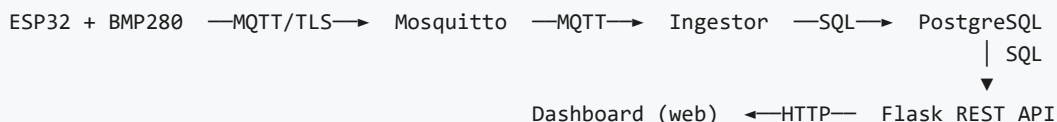


Rozproszone Systemy Pomiarowe — Dokumentacja projektu

Rozproszony system pomiarowy zbierający dane z czujników **BMP280** (temperatura, ciśnienie) podłączonych do **ESP32**, przesyłający je przez **MQTT** do backendu, który waliduje, zapisuje do **PostgreSQL** i udostępnia przez **REST API (Flask)**. Warstwą prezentacji jest **dashboard webowy w Streamlit**.



Bezpieczeństwo (skrót). Cały ruch sieciowy jest szyfrowany:

- **MQTT po TLS na porcie 8883** — ESP32 łączy się z brokerem przez sieć Wi-Fi wyłącznie szyfrowanym kanałem (własne CA, weryfikacja tożsamości brokera).
- **Port 1883 (plaintext) istnieje wyłącznie wewnątrz sieci Docker** (ingestor → broker) i **nie jest wystawiony na hosta** — jest nieosiągalny spoza kontenerów.
- **REST API chronione HTTP Basic Auth** — endpointy z danymi wymagają logowania.
- **Izolacja usług** — baza i wewnętrzny broker bez mapowania portów na hosta.

Ten plik to **scalona, jednoplikowa wersja** całej dokumentacji (wersja do druku / PDF). Modułowe źródła znajdują się w katalogu [docs/](#).

Spis treści

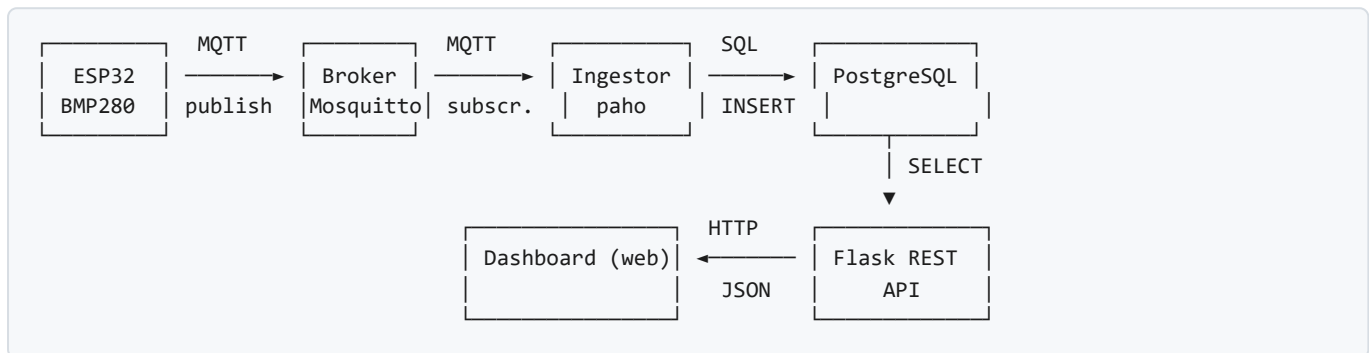
1. Architektura systemu
2. Uruchomienie krok po kroku
3. Firmware ESP32 — pomiar i publikacja
4. Kontrakt komunikacji MQTT
5. Ingestor — MQTT → PostgreSQL
6. Baza danych (PostgreSQL)
7. REST API (Flask)
8. Uwierzytelnianie API — HTTP Basic Auth
9. Dashboard webowy (Streamlit)
10. Niezawodność ESP32 — reconnect, status, LWT
11. Bezpieczeństwo — TLS na MQTT
12. Dodatek: LabVIEW UI (archiwum)
13. Test end-to-end
14. Status realizacji (laboratoria)

1. Architektura systemu

Przegląd

Rozproszony system pomiarowy, w którym urządzenia ESP32 z czujnikami BMP280 zbierają dane (temperatura, ciśnienie), publikują je przez MQTT, a warstwa backendowa odbiera, waliduje, zapisuje do bazy i udostępnia przez REST API.

Przepływ danych end-to-end



Warstwy systemu

1. Warstwa urządzeń brzegowych — ESP32

- Mikrokontroler ESP32 z czujnikiem **BMP280** (I2C, adres 0x76).
- Mierzy temperaturę (°C) i ciśnienie (hPa) co 5 sekund.
- Generuje stabilny `device_id` z eFuse MAC: `esp32-XXXXXXXXXX`.
- Synchronizuje czas przez NTP (`time.cloudflare.com`, `time.google.com`).
- Publikuje do dwóch topiców MQTT (osobny dla każdego sensora).
- Auto-reconnect Wi-Fi/MQTT z backoff 2 s.

2. Warstwa komunikacji — MQTT (Mosquitto)

- Broker Eclipse Mosquitto — dwa listenery: `1883` (plaintext, tylko wewnątrz sieci Docker) oraz `8883` (TLS, dla ESP32 i klientów zewnętrznych).
- Konfiguracja: `allow_anonymous true`; TLS z własnym CA (lab 10 — patrz `security_tls.md`).
- Persystencja włączona (`/mosquitto/data/`).
- Topic pattern: `lab/<group_id>/<device_id>/<sensor>`.

3. Warstwa zbierania danych — Ingestor

- Serwis w Pythonie (`paho-mqtt`).
- Subskrybuje wildcard `lab/+/+/+`.
- Waliduje obecność wymaganych pól (`device_id`, `sensor`, `value`, `ts_ms`).
- Zapisuje poprawne wiadomości do PostgreSQL przez `psycopg2`.
- Odrzuca i loguje wiadomości niezgodne z kontraktem.

4. Warstwa przechowywania — PostgreSQL 18

- Dwie tabele: `sensor` (metadane urządzeń) i `measurements` (pomiar).
- Inicjalizacja przez `database/01-init_database.sql` przy pierwszym starcie.
- Port `5432` — tylko wewnątrz sieci Docker (po lab 10 nie mapowany na host).

5. Warstwa dostępu — Flask REST API

- Serwer Flask (port `5001`).
- Endpointy odczytowe (GET): `/health`, `/devices`, `/latest`, `/history`, `/latest/temperature`.
- Zwraca JSON.
- Filtrowanie po `device_id`, `sensor`, `limit`.

6. Warstwa prezentacji — Dashboard webowy (Streamlit)

- Aplikacja Streamlit (`wykresy_python/`) — kafelki aktualnych pomiarów, wykresy trendu (Plotly), historia + eksport CSV, filtr dat, auto-odświeżanie.
- Klient REST — czyta z Flask przez HTTP, nie zna schematu bazy.
- Działa **poza Dockerem**. Szczegóły: `wykresy.md`.
- LabVIEW (`ui/`) — poprzednia wersja prezentacji, w archiwum (`labview.md`).

Konteneryzacja

Wszystkie serwisy backendowe uruchamiane przez Docker Compose. Cztery kontenery:

Kontener	Obraz	Port	Zależności
<code>broker</code>	<code>eclipse-mosquitto</code>	8883 TLS (+1883 wewn.)	—
<code>postgres</code>	<code>postgres:18-alpine</code>	5432 (wewn.)	—
<code>ingestor</code>	<code>python:3.10-slim</code>	—	broker, database
<code>api</code>	<code>python:3.10-slim</code>	5001	database

Każdy kontener buduje się z własnego `Dockerfile` w odpowiednim katalogu. ESP32 i dashboard (Streamlit) NIE są w Dockerze (urządzenie fizyczne / klient REST w przeglądarce).

Decyzje architektoniczne

Dlaczego ESP nie pisze bezpośrednio do bazy? Rozdzielenie warstw — urządzenie nie zna struktury bazy, ingestor centralizuje walidację, kontrakt MQTT jest stabilnym interfejsem. Łatwiej dołożyć kolejne urządzenia lub kolejnego subskrybenta (np. drugi ingestor do innej bazy).

Dlaczego prezentacja przez REST, a nie przez SQL? REST ukrywa schemat bazy, daje stabilny kontrakt, pozwala filtrować po stronie backendu. Zmiana schematu bazy nie wymaga zmian w UI.

Dlaczego osobne topiki dla `temperature` i `pressure`? Pozwala subskrybować selektywnie (np. tylko temperatury z wszystkich urządzeń: `lab/+/+/temperature`). Spójne z kontraktem `lab/<grp>/<dev>/<sensor>`.

2. Uruchomienie krok po kroku

Wymagania

- **Docker Desktop** + **Docker Compose** (Windows: WSL2 backend).
- **PlatformIO** (VS Code extension) do firmware ESP32.
- **MQTT Explorer** (opcjonalnie, do podglądu wiadomości — po lab 10 z TLS).
- Porty `8883` (MQTT/TLS) i `5001` (API) wolne na hoście. Porty `1883` i `5432` działają tylko wewnątrz sieci Docker (nie są mapowane na host).

Backend — Docker Compose

Konfiguracja

W katalogu głównym skopiuj `.env.example` jako `.env` i uzupełnij:

```
DB_HOST=postgres
DB_NAME=abcd_db
DB_USER=admin
DB_PASSWORD=admin_pass1234
```

Uwaga: `DB_HOST=postgres` to nazwa kontenera w sieci Compose — nie zmieniaj. Hasło można zmienić, ale musi być spójne z `.env`.

Start

```
# Pierwszy raz – z logami w foreground (widać błędy startu)
docker compose up --build

# Później – w tle
docker compose up -d --build

# Status
docker compose ps

# Logi
docker compose logs -f
docker compose logs -f ingestor
docker compose logs -f api
docker compose logs -f broker
docker compose logs -f database
```

Stop

```
docker compose down          # zatrzymaj
docker compose down -v       # zatrzymaj + usuń wolumeny (kasuje bazę!)
```

Weryfikacja działania

Broker MQTT

W MQTT Explorer:

- Host: `localhost` , Port: `8883` , włącz **Encryption (TLS)** i wczytaj `certs/ca.crt` (po lab 10 broker wymaga TLS — patrz `security_tls.md`).
- Powinno połączyć się i pokazać drzewo topiców `$SYS` .

Baza PostgreSQL

```
docker exec -it postgres psql -U admin -d abcd_db
```

W psql:

```
\dt -- lista tabel
SELECT COUNT(*) FROM measurements;
SELECT * FROM measurements ORDER BY received_at DESC LIMIT 5;
\q
```

REST API

W przeglądarce lub przez `curl` :

```
curl http://localhost:5001/ # "Hello, World!"
curl http://localhost:5001/health # {"status":"ok"}
curl http://localhost:5001/devices # lista device_id
curl http://localhost:5001/latest # ostatni pomiar per (device,sensor)
curl http://localhost:5001/latest/temperature # ostatnia temperatura per urządzenie
curl "http://localhost:5001/history?limit=10" # historia (DESC)
```

Ingestor

Powinien w logach pisać:

```
[MQTT] Połączono z brokerem, rc=0
[MQTT] Subskrypcja: lab/+/+/+
[OK] Zapisano z topicu: lab/g03/esp32-XXXX/temperature # po każdej wiadomości
```

Dashboard webowy (Streamlit)

Dashboard działa **poza Dockerem** — uruchom po starcie backendu:

```
cd wykresy_python

# Pierwsze uruchomienie – zainstaluj zależności
python -m pip install -r requirements.txt

# Uruchom
python -m streamlit run app.py
```

Przeglądarka otworzy się automatycznie pod `http://localhost:8501` . W pasku bocznym powinien być widoczny status **Backend: online ✓**.

Jeśli chcesz wskazać inny adres backendu (np. IP koleżanki w sieci lokalnej), zmień pole **Base URL backendu** w pasku bocznym — bez restartu.

Szczegóły i rozwiązywanie problemów: `docs/wykresy.md` .

Firmware ESP32

Konfiguracja

W `esp32/` skopiuj `secrets.h.example` jako `include/secrets.h` (UWAGA — plik musi trafić do `include/`, nie obok `secrets.h.example`):

```
#define WIFI_SSID      "NAZWA_WIFI_LAB"
#define WIFI_PASSWORD  "HASLO_WIFI"
#define MQTT_HOST      "192.168.X.Y"    // IP hosta z Dockerem (sprawdź ipconfig)
#define MQTT_PORT      8883             // TLS (lab 10)
#define MQTT_GROUP      "g03"           // numer grupy laboratoryjnej
```

Klucz: `MQTT_HOST` to IP komputera z Dockerem w sieci laboratoryjnej, NIE `localhost` (ESP32 nie jest na tym samym hoście co broker). Na Windows sprawdzasz przez `ipconfig` w sekcji "Wireless LAN adapter Wi-Fi → IPv4 Address".

Sprzęt — BMP280 → ESP32

BMP280	ESP32
VCC	3.3V
GND	GND
SCL	GPIO 22
SDA	GPIO 21
CSB	GPIO 23 (HIGH w <code>setup()</code> — wymusza I2C)
SDO	GND (adres I2C: 0x76)

Build i upload

W VS Code z PlatformIO:

1. **Build** (ikona ✓ na pasku PlatformIO).
2. Podłącz ESP32 przez USB (powinien się wykryć na COM3 — sprawdź `platformio.ini`, w razie potrzeby zmień `upload_port` i `monitor_port`).
3. **Upload** (ikona →).
4. **Serial Monitor** — powinno się pojawić:

```
BMP280 gotowy.
Device ID: esp32-A1B2C3D4E5F6
Laczenie z Wi-Fi: NAZWA_WIFI
...
Polaczono z Wi-Fi
Adres IP: 192.168.X.Z
Synchronizacja NTP...
Czas zsynchronizowany.
Laczenie z MQTT...OK
Publikacja: {"schema_version":1,"group_id":"g03",...,"sensor":"temperature","value":24.3,...}
```

Test end-to-end

1. Uruchom Docker Compose: `docker compose up -d`.
2. Wgraj firmware na ESP32 (skonfigurowany `secrets.h`).
3. ESP32 publikuje co 5 s → broker → ingestor → baza.
4. Sprawdź logi ingestora (`docker compose logs -f ingestor`) — powinno lecieć `[OK] Zapisano...`.
5. Sprawdź bazę:

```
docker exec -it postgres psql -U admin -d abcd_db \  
-c "SELECT device_id, sensor, value, received_at FROM measurements ORDER BY id DESC LIMIT 5;"
```

6. Sprawdź API: `curl http://localhost:5001/latest`.
7. Uruchom dashboard: `cd wykresy_python && python -m streamlit run app.py`. Pod `http://localhost:8501` powinny być widoczne aktualne kafelki i wykresy.

Typowe problemy

Cannot connect to the Docker daemon — Docker Desktop nie działa lub WSL nie zintegrowany. W Docker Desktop: Settings → Resources → WSL Integration.

port is already allocated — port 8883/5001 zajęty. Sprawdź `netstat -ano | findstr :8883` i zabij proces albo zmień mapowanie portów w `docker-compose.yml`.

Ingestor restartuje się w pętli — baza jeszcze się nie zainicjalizowała, ingestor próbuje połączyć się za wcześniej. `restart: on-failure` to obsłuży, poczekaj 10-20 s.

ESP32 nie łączy się z MQTT — sprawdź `MQTT_HOST` (musi być IP hosta widoczne z sieci ESP, nie `localhost`). Sprawdź firewall Windows — port 8883 musi być otwarty dla sieci lokalnej.

Nie znaleziono BMP280! — sprawdź połączenia I2C (SDA/SCL), zasilanie 3.3V, czy CSB jest na HIGH (wymusza I2C zamiast SPI), czy SDO na GND (adres 0x76 zamiast 0x77).

Wiadomości publikują się, ale ingestor nic nie zapisuje — sprawdź czy topic pasuje do wzorca `lab/+/+/+` (musi mieć dokładnie 4 segmenty). Sprawdź czy payload ma wszystkie 4 wymagane pola.

3. Firmware ESP32 — pomiar i publikacja

Co robi

Cyklicznie (co 5 s) odczytuje z BMP280 temperaturę i ciśnienie, opakuje je w JSON i publikuje do brokera MQTT na osobnych topicach. Obsługuje reconnect Wi-Fi i MQTT, synchronizuje czas przez NTP.

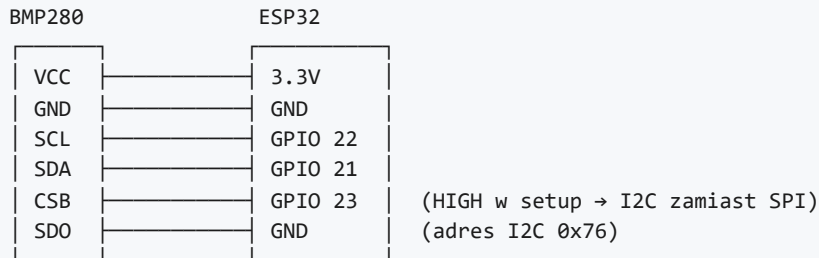
Stos

- **Płytki:** ESP32 Dev Module (PlatformIO `board = esp32dev`).
- **Framework:** Arduino.
- **Biblioteki** (`platformio.ini`):
 - `knolleary/PubSubClient` — klient MQTT.
 - `bblanchon/ArduinoJson` — serializacja JSON.

- `adafruit/Adafruit BMP280 Library` — sterownik czujnika.

Sprzęt

BMP280 → ESP32 (I2C)



CSB i SDO są kluczowe:

- `CSB = HIGH` → BMP280 pracuje w trybie I2C (nie SPI).
- `SDO = GND` → adres I2C = `0x76` (przy `SDO = VCC` byłby `0x77`).

Główne funkcje

`generateDeviceIdFromEfuse()`

Czyta MAC z eFuse (`ESP.getEfuseMac()`) i formatuje na string `esp32-xxxxxxxxxxxx`. Stabilny między restartami, unikalny per egzemplarz — ten sam firmware na różnych płytkach generuje różne `device_id`.

```
String generateDeviceIdFromEfuse() {
    uint64_t chipId = ESP.getEfuseMac();
    char id[32];
    snprintf(id, sizeof(id), "esp32-%04X%08X",
        (uint16_t)(chipId >> 32),
        (uint32_t)chipId);
    return String(id);
}
```

Alternatywa wg Lab 3 — UUIDv4 zapisany w NVS przy pierwszym starcie. Tu celowo wariant prostszy, bo MAC-based ID wystarcza do unikalności.

`connectWiFi()`

Łączy z siecią z `secrets.h`. Blokuje aż `WL_CONNECTED`. Po połączeniu wypisuje IP.

`syncNTP()`

`configTime(3600, 3600, "time.cloudflare.com", "time.google.com")` — UTC+1 z DST (CET/CEST). Blokuje aż `getLocalTime()` zwróci sukces. Bez tego `ts_ms` byłby względny do startu układu, nie do epoki Unix.

Uwaga: w przykładowym `main.txt` (wzór z instrukcji) NTP używa `configTime(0, 0, "pool.ntp.org", "time.nist.gov")` — UTC bez offsetu. Wersja w `main.cpp` używa CET/CEST. Konsekwencja: `ts_ms` w bazie zawiera czas lokalny, nie UTC. Jeśli planujesz wiele stref czasowych — przejść na UTC i konwertować przy prezentacji.

getTimestampMs()

Zwraca aktualny czas jako Unix epoch w ms:

```
long long getTimestampMs() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return ((long long)tv.tv_sec * 1000LL) + (tv.tv_usec / 1000);
}
```

connectMQTT()

Pętla retry z `delay(2000)` — łączy do `MQTT_HOST:MQTT_PORT` używając `deviceId` jako Client ID. Jako Client ID użycie `deviceId` zapewnia unikalność klientów na brokerze (broker rozłącza poprzedniego z tym samym ID).

publishMeasurement()

Wysła **dwie wiadomości** za każdym wywołaniem (jedna na temperaturę, druga na ciśnienie), z dwoma kolejnymi `seq` (inkrementowany po każdej publikacji). Każda na własny topic:

- `lab/<MQTT_GROUP>/<deviceId>/temperature`
- `lab/<MQTT_GROUP>/<deviceId>/pressure`

Topiki obliczane raz w `setup()` jako globalne `topicTemp` i `topicPressure` — nie ma alokacji w pętli.

loop()

```
void loop() {
    if (WiFi.status() != WL_CONNECTED) { connectWiFi(); syncNTP(); }
    if (!mqttClient.connected()) { connectMQTT(); }
    mqttClient.loop();
    publishMeasurement();
    delay(5000);
}
```

Co iterację: sprawdza Wi-Fi, sprawdza MQTT, obsługuje pętlę MQTT, publikuje, czeka 5 s. `mqttClient.loop()` jest wymagane przez PubSubClient do obsługi keep-alive i ewentualnych callbacków.

Format wiadomości

Pełny opis: `message_contract.md`.

```
{
  "schema_version": 1,
  "group_id": "g03",
  "device_id": "esp32-F88DAB004F8C",
  "sensor": "temperature",
  "value": 24.5,
  "unit": "C",
  "ts_ms": 1774285098907,
  "seq": 0
}
```

Konfiguracja — secrets.h

Plik **NIE jest w repo** (`.gitignore`: `esp32/include/secrets.h`). Skopiuj wzorzec i uzupełnij:

```
cp esp32/secrets.h.example esp32/include/secrets.h
```

Uwaga na ścieżkę — w repo wzorzec leży w `esp32/secrets.h.example`, ale sam `secrets.h` musi być w `esp32/include/secrets.h` (PlatformIO szuka nagłówków w `include/`).

```
#define WIFI_SSID      "..."  
#define WIFI_PASSWORD "..."  
#define MQTT_HOST     "192.168.X.Y" // IP HOSTA Z DOCKEREM, nie localhost!  
#define MQTT_PORT     8883          // TLS (lab 10)  
#define MQTT_GROUP    "g03"
```

Build i flash

```
# Z PlatformIO CLI  
pio run                                # build  
pio run --target upload                # flash  
pio device monitor                    # serial monitor (115200 baud)  
  
# Lub w VS Code: paszek PlatformIO → Build / Upload / Monitor
```

`upload_port = COM3` w `platformio.ini` — jeśli ESP32 jest na innym COM, zmień lub usuń (auto-detect).

Co można dodać

- **Last Will Testament** — zarejestrować retained message na `lab/<grp>/<dev>/status` z payloadem `{"status":"offline"}`. Broker opublikuje to automatycznie gdy ESP32 zerwie połączenie (Lab 9 — niezawodność).
- **Buforowanie offline** — bufor w RAM (kilka pomiarów) na wypadek utraty połączenia, wysyłka po reconnect.
- **QoS 1** — obecnie domyślne QoS 0 (fire-and-forget). QoS 1 daje gwarancję dostarczenia (ale kosztuje pamięć i przepustowość).
- **Sygnalizacja LED** — diody jako wskaźnik stanu (Wi-Fi / MQTT / sensor).
- **NVS dla device_id** — wariant z Lab 3 (UUIDv4 zapisywany przy pierwszym starcie). Obecnie ID jest deterministyczne z MAC, więc nie ma realnej potrzeby.

4. Kontrakt komunikacji MQTT

Pełna kopia kontraktu, który jest też w `esp32/docs/message_contract.md`. Tutaj dla wygody — w docs/projektu zgodnie z konwencją z laboratoriów.

Struktura topicu

```
lab/<group_id>/<device_id>/<sensor>
```

Cztery segmenty rozdzielone `/`. Każde urządzenie publikuje **osobną wiadomość na każdy typ sensora** (nie jedną wiadomość ze wszystkimi wartościami).

Przykład:

```
lab/g03/esp32-F88DAB004F8C/temperature
lab/g03/esp32-F88DAB004F8C/pressure
```

Zasady nazewnictwa segmentów

- Małe litery, bez spacji, bez polskich znaków.
- `group_id` — `g` + 2 cyfry (`g01` , `g03` ...).
- `device_id` — `esp32-` + 12 znaków hex z MAC.
- `sensor` — krótki tekst (`temperature` , `pressure` , `humidity` , ...).

Payload — wiadomość pomiarowa JSON

```
{
  "schema_version": 1,
  "group_id": "g03",
  "device_id": "esp32-F88DAB004F8C",
  "sensor": "temperature",
  "value": 24.5,
  "unit": "C",
  "ts_ms": 1742030400000,
  "seq": 15
}
```

Pola

Wymagane

Pole	Typ	Opis
<code>device_id</code>	string	Unikalny identyfikator urządzenia (niepusty)
<code>sensor</code>	string	Typ sensora (niepusty)
<code>value</code>	number	Wartość pomiaru (int lub float)
<code>ts_ms</code>	integer	Czas pomiaru — Unix epoch w milisekundach

Opcjonalne (zalecane)

Pole	Typ	Opis
<code>schema_version</code>	integer	Wersja kontraktu (obecnie <code>1</code>)
<code>group_id</code>	string	Identyfikator grupy laboratoryjnej
<code>unit</code>	string	Jednostka fizyczna (<code>"C"</code> , <code>"hPa"</code> , <code>"%"</code>)
<code>seq</code>	integer	Numer sekwencyjny wiadomości z urządzenia

Reguły walidacji

W ingestorze (`ingestor/ingestor.py`):

```
REQUIRED_FIELDS = ["device_id", "sensor", "value", "ts_ms"]

def is_valid(data):
    return all(field in data for field in REQUIRED_FIELDS)
```

Walidowana jest **tylko obecność** pól. Typy nie są sprawdzane — błąd typu (`value` jako string) wyleci w `INSERT` do PostgreSQL i trafi do `[ERR]`, nie do `[SKIP]`.

Pełniejsza walidacja, której można się trzymać przy ręcznych testach:

- `device_id` — niepusty string.
- `sensor` — niepusty string.
- `value` — liczba (int lub float), nie string.
- `ts_ms` — dodatnia liczba całkowita (Unix epoch ms — po roku 2001).
- `unit` — jeśli podany, spójny z `sensor` (`temperature` → `"C"`, `pressure` → `"hPa"`).
- `seq` — jeśli podany, nieujemna liczba całkowita.

Mapowanie na schemat bazy

Pole JSON	Kolumna w measurements	Uwagi
<code>device_id</code>	<code>device_id</code>	NOT NULL
<code>sensor</code>	<code>sensor</code>	NOT NULL
<code>value</code>	<code>value</code>	NOT NULL, DOUBLE PRECISION
<code>ts_ms</code>	<code>ts_ms</code>	NOT NULL, BIGINT
<code>schema_version</code>	— (ignorowane)	Pole walidacyjne klienta
<code>group_id</code>	<code>group_id</code>	NULL jeśli brak
<code>unit</code>	<code>unit</code>	NULL jeśli brak
<code>seq</code>	<code>seq</code>	NULL jeśli brak
(topic MQTT)	<code>topic</code>	Pełna ścieżka topicu
(czas serwera)	<code>received_at</code>	DEFAULT CURRENT_TIMESTAMP

Przykłady

Poprawna wiadomość

```
{
  "schema_version": 1,
  "group_id": "g03",
  "device_id": "esp32-F88DAB004F8C",
  "sensor": "temperature",
  "value": 24.5,
  "unit": "C",
  "ts_ms": 1774285098907,
  "seq": 1
}
```

Topic: `lab/g03/esp32-F88DAB004F8C/temperature` → zapis OK.

Wiadomość błędna — brak `ts_ms`

```
{
  "device_id": "esp32-F88DAB004F8C",
  "sensor": "temperature",
  "value": 24.5,
  "unit": "C"
}
```

Ingestor: `[SKIP]` Brak wymaganych pól. Nie zapisuje.

Wiadomość błędna — `value` jako string

```
{
  "device_id": "esp32-F88DAB004F8C",
  "sensor": "temperature",
  "value": "24.5",
  "unit": "C",
  "ts_ms": 1774285098907
}
```

Ingestor: `is_valid()` → True, ale `INSERT` zawiedzie (PostgreSQL nie sparsuje stringa do `DOUBLE PRECISION`).
Trafia do `[ERR]`.

Wiadomość błędna — niepoprawny JSON

```
{ device_id: "esp32-...", value: 24.5 }
```

(brak cudzysłowów wokół kluczy)

Ingestor: `[ERR]` Expecting property name enclosed in double quotes.

Wersjonowanie

Pole `schema_version` pozwala na ewolucję kontraktu bez łamania starych klientów:

- v1 (obecna) — opisana wyżej.
- v2 (potencjalna) — np. `ts` jako ISO 8601 zamiast `ts_ms`, lub `meta: {...}` na dodatkowe pola.

Ingestor obecnie **nie patrzy** na `schema_version` — przyjmuje wszystko co ma cztery wymagane pola. Pole jest tylko informacyjne.

Topiki specjalne (proponowane, nie zaimplementowane)

Lab 4 dodatkowo sugeruje rozdzielenie wiadomości statusowych:

```
lab/<group_id>/<device_id>/status
```

Payload:

```
{
  "schema_version": 1,
  "device_id": "esp32-F88DAB004F8C",
  "status": "online",
  "ts_ms": 1742030400000
}
```

Naturalne miejsce do podłączenia **Last Will Testament** w MQTT — broker opublikuje retained `{"status": "offline"}` gdy urządzenie zerwie połączenie. Wymaga rozszerzenia ingestora (rozdzielenie sensora pomiarowego od statusu).

5. Ingestor — MQTT → PostgreSQL

Opis

Serwis backendowy w Pythonie, który subskrybuje wszystkie pomiary z brokera MQTT, waliduje strukturę wiadomości i zapisuje poprawne dane do PostgreSQL. Odpowiedzialny za **walidację kontraktu** — wiadomości bez wymaganych pól są odrzucane z logiem `[SKIP]`.

Stos

- **Python 3.10-slim** (Dockerfile).
- **paho-mqtt 1.6.1** — klient MQTT.
- **psycopg2-binary 2.9.9** — sterownik PostgreSQL.

Struktura

```
ingestor/
├── ingestor.py    # główna logika: MQTT + walidacja + zapis
├── db.py          # połączenie z PostgreSQL
├── requirements.txt
└── Dockerfile
```

Konfiguracja

Połączenia są skonfigurowane przez **zmienne środowiskowe** (z `.env` w Compose) z fallbackami w `db.py`:

Zmienna	Domyślnie (fallback)
<code>DB_HOST</code>	<code>postgres</code>
<code>DB_NAME</code>	<code>abcd_db</code>
<code>DB_USER</code>	<code>admin</code>
<code>DB_PASSWORD</code>	<code>admin_pass1234</code>

Adres brokera jest **na sztywno** w `ingestor.py`:

```
MQTT_HOST = "broker"    # nazwa kontenera w Compose
MQTT_PORT = 1883
MQTT_TOPIC = "lab/+/+/+
```

Działanie

Subskrybowany wzorzec topiców

```
lab/+/+/+
```

Pasuje do każdego `lab/<grupa>/<urządzenie>/<sensor>` — np.:

- `lab/g03/esp32-F88DAB004F8C/temperature` ✓
- `lab/g03/esp32-F88DAB004F8C/pressure` ✓
- `lab/g03/esp32-F88DAB004F8C/status/online` ✗ (5 segmentów)
- `lab/g03/esp32-F88DAB004F8C` ✗ (3 segmenty)

Pola wymagane (walidacja)

```
REQUIRED_FIELDS = ["device_id", "sensor", "value", "ts_ms"]
```

Wszystkie muszą być obecne. Wartości nie są walidowane typowo (string vs liczba) — to obowiązek nadawcy.

Pola opcjonalne (zapisywane jeśli są)

`schema_version`, `group_id`, `unit`, `seq`. Pobierane przez `data.get(...)` — jeśli brak, do bazy trafi `NULL`.

Pętla zdarzeniowa

Wzorzec callbacków paho-mqtt:

```
def on_connect(client, userdata, flags, rc):
    client.subscribe(MQTT_TOPIC)

def on_message(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode("utf-8"))
        if not is_valid(data):
            print(f"[SKIP] Brak wymaganych pol: ...")
            return
        save_measurement(msg.topic, data)
        print(f"[OK] Zapisano z topicu: {msg.topic}")
    except Exception as e:
        print(f"[ERR] {e}")

client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message
client.connect(MQTT_HOST, MQTT_PORT, 60)
client.loop_forever()
```

`loop_forever()` blokuje główny wątek i obsługuje połączenie + reconnect.

Zapis do bazy

```
INSERT INTO measurements
  (group_id, device_id, sensor, value, unit, ts_ms, seq, topic)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
```

Każda wiadomość = jeden `INSERT` z commit. **Nie ma batchowania** — przy dużym ruchu można optymalizować przez `executemany` lub buforowanie.

Topic jest zapisywany jako dodatkowa kolumna — pozwala śledzić skąd wiadomość przyszła, nawet jeśli `group_id` w payloadzie był pomyłony lub brakujący.

Uruchomienie

Razem z Compose:

```
docker compose up -d --build ingestor
docker compose logs -f ingestor
```

Sam (do debugowania, wymaga lokalnej bazy i brokera):

```
cd ingestor
pip install -r requirements.txt
python ingestor.py
```

Test

Publikacja testowa przez mosquitto_pub

Po lab 10 port `1883` nie jest wystawiony na hosta (tylko wewnątrz sieci Docker). Testy uruchamiamy **wewnątrz kontenera brokera** przez `docker exec` (wewnętrzny listener plaintext). Z hosta alternatywnie po TLS: `-p 8883 --cafile certs/ca.crt`.

```
docker exec broker mosquitto_pub -h localhost -p 1883 \
  -t "lab/g03/esp32-test/temperature" \
  -m '{"device_id":"esp32-test","sensor":"temperature","value":24.5,"unit":"C","ts_ms":1742030400000}'
```

Oczekiwane:

- Log ingestora: `[OK] Zapisano z topicu: lab/g03/esp32-test/temperature`.
- W bazie nowy rekord (`SELECT * FROM measurements ORDER BY id DESC LIMIT 1`).

Test wiadomości błędnej

```
docker exec broker mosquitto_pub -h localhost -p 1883 \
  -t "lab/g03/esp32-test/temperature" \
  -m '{"device_id":"esp32-test","sensor":"temperature","value":24.5}'
```

(brak `ts_ms`)

Oczekiwane:

- Log ingestora: [SKIP] Brak wymaganych pol: ...
- Brak rekordu w bazie.

Test złamanego JSON-a

```
docker exec broker mosquitto_pub -h localhost -p 1883 \
  -t "lab/g03/esp32-test/temperature" \
  -m 'not a json'
```

Oczekiwane:

- Log ingestora: [ERR] Expecting value: line 1 column 1 (char 0).

Znane ograniczenia

- **Nowe połączenie z DB per wiadomość** — `save_measurement` otwiera i zamyka połączenie za każdym razem. Przy dużym ruchu to wąskie gardło. Do poprawy: pool połączeń (`psycopg2.pool.SimpleConnectionPool`) albo jedno długie połączenie z `reconnect` przy błędzie.
- **Brak walidacji typów** — `value` jako string przejdzie walidację `is_valid` ale zawiedzie w `INSERT (DOUBLE PRECISION)`. Trafi do [ERR], nie do [SKIP].
- **Brak idempotencji** — duplikaty wiadomości (np. po `reconnect MQTT` przy QoS 1) tworzą duplikat rekordów. Możliwe rozwiązanie: `UNIQUE (device_id, sensor, ts_ms, seq) + ON CONFLICT DO NOTHING`.
- **Brak keep-alive długiego** — domyślne 60 s. Przy bardzo niestabilnej sieci może to być za długo do wykrycia rozłączenia.
- **Hardkodowany MQTT_HOST = "broker"** — działa tylko w Compose. Do uruchomienia poza Compose: refaktor na `os.getenv("MQTT_HOST", "broker")`.

6. Baza danych (PostgreSQL)

Konfiguracja

- **Obraz:** `postgres:18-alpine` (Dockerfile w `database/`).
- **Port:** `5432` — tylko wewnątrz sieci Docker (po lab 10 nie mapowany na host).
- **Dane logowania:** z `.env` (`DB_USER`, `DB_PASSWORD`, `DB_NAME`).
- **Inicjalizacja:** `database/01-init_database.sql` uruchamia się automatycznie przy pierwszym starcie kontenera (mechanizm `/docker-entrypoint-initdb.d/`).

Schemat

Tabela `measurements` — pomiary

Główna tabela systemu. Każdy poprawny pomiar z MQTT trafia jako jeden rekord.

```
CREATE TABLE IF NOT EXISTS measurements (
  id          SERIAL PRIMARY KEY,
  group_id    TEXT,
  device_id   TEXT NOT NULL,
  sensor      TEXT NOT NULL,
  value       DOUBLE PRECISION NOT NULL,
  unit        TEXT,
  ts_ms       BIGINT NOT NULL,
  seq         INTEGER,
  topic       TEXT,
  received_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Kolumna	Typ	Opis
id	SERIAL PK	Sztuczny klucz, auto-inkrement
group_id	TEXT	Grupa laboratoryjna (np. g03)
device_id	TEXT NOT NULL	Identyfikator urządzenia (esp32-XXXX...)
sensor	TEXT NOT NULL	Typ czujnika (temperature , pressure)
value	DOUBLE PRECISION	Wartość pomiaru
unit	TEXT	Jednostka (C , hPa)
ts_ms	BIGINT NOT NULL	Czas pomiaru z urządzenia (Unix epoch, ms)
seq	INTEGER	Numer sekwencyjny wiadomości z urządzenia
topic	TEXT	Pełny topic MQTT, z którego przyszła wiadomość
received_at	TIMESTAMP DEFAULT	Czas odebrania przez ingestor (czas serwera)

Dwa znaczniki czasu:

- ts_ms — czas pomiaru na urządzeniu (po synchronizacji NTP).
- received_at — czas odebrania na backendzie.

Różnica między nimi to **opóźnienie sieciowe + przetwarzanie**. Pozwala diagnozować problemy (urządzenie z rozjechanym zegarem, gubione wiadomości, opóźniony reconnect).

Tabela sensor — metadane urządzeń

```
CREATE TABLE sensor (
  uuid        UUID PRIMARY KEY,
  name        TEXT NOT NULL,
  type        TEXT,
  sensor      TEXT,
  apidetails  TEXT,
  is_online   BOOLEAN DEFAULT FALSE
);
```

Status: tabela zdefiniowana, ale obecnie **nieużywana** przez ingestor ani API. Przewidziana do przyszłej rozbudowy (rejestracja urządzeń, status online/offline z Last Will Testament).

Przykładowe zapytania

Lista wszystkich urządzeń

```
SELECT DISTINCT device_id FROM measurements ORDER BY device_id;
```

Ostatni pomiar każdego (urządzenie, sensor)

```
SELECT DISTINCT ON (device_id, sensor)
    device_id, sensor, value, unit, ts_ms, received_at
FROM measurements
ORDER BY device_id, sensor, received_at DESC;
```

Historia ostatnich 50 pomiarów temperatury

```
SELECT device_id, value, unit, ts_ms, received_at
FROM measurements
WHERE sensor = 'temperature'
ORDER BY received_at DESC
LIMIT 50;
```

Statystyki per urządzenie

```
SELECT device_id, sensor,
    COUNT(*) as n,
    MIN(value) as min, MAX(value) as max, AVG(value) as avg,
    MAX(received_at) as last_seen
FROM measurements
GROUP BY device_id, sensor
ORDER BY device_id, sensor;
```

Opóźnienie urządzenie → serwer (s)

```
SELECT device_id,
    AVG(EXTRACT(EPOCH FROM received_at) - ts_ms/1000.0) as avg_delay_s
FROM measurements
WHERE ts_ms > 1700000000000 -- po synchronizacji NTP
GROUP BY device_id;
```

Dostęp z hosta

```
# Przez kontener (najprostsze)
docker exec -it postgres psql -U admin -d abcd_db

# Z hosta (np. DBeaver, pgAdmin, VS Code SQLTools):
# po lab 10 port 5432 NIE jest mapowany na host. Aby połączyć narzędziem z hosta,
# tymczasowo dodaj `ports: ["5432:5432"]` do usługi `database` w docker-compose.yml
# (albo korzystaj z `docker exec ... psql` powyżej).
```

Persystencja

Compose **nie definiuje** wolumenu dla danych Postgresa — przy `docker compose down -v` lub przebudowie obrazu dane znikają. Do zachowania danych między restartami dodaj wolumen w `docker-compose.yml`:

```
database:
  ...
  volumes:
    - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Co warto dodać (znane luki)

- **Indeks** na `(device_id, sensor, received_at DESC)` — przyspieszy zapytania o historię i ostatni pomiar (obecnie tylko PK na `id`).
- **Indeks** na `received_at DESC` — przyspieszy `/history`.
- **UNIQUE** na `(device_id, sensor, ts_ms, seq)` — odporność na duplikaty (re-publicacja po reconnect MQTT może wysłać tę samą wiadomość dwa razy).
- **Wolumen** dla persystencji.
- **Wykorzystanie tabeli** `sensor` lub jej usunięcie.

7. REST API (Flask)

Opis

Serwer Flask udostępniający dane pomiarowe z PostgreSQL przez HTTP/JSON. Klient (dashboard Streamlit, przeglądarka, curl) wykonuje żądania GET, otrzymuje JSON.

Stos

- **Flask** (najnowsza z pip, bez pin).
- **psycopg2-binary** — dostęp do PostgreSQL.
- **Python 3.10-slim** (Dockerfile).

Konfiguracja

- Port: `5001` (w `app.run(..., port=5001)` i mapowanie w Compose).
- Bind: `0.0.0.0` (dostępne z hosta).
- Tryb: `debug=True` (dev — w produkcji wyłączyć!).
- Połączenie z bazą: jak w ingestorze, przez zmienne `DB_*` z `.env`.

Endpointy

Wszystkie odpowiadają na **GET**, wszystkie zwracają **JSON**. Base URL: `http://localhost:5001`.

GET /

Sanity check (HTML).

Odpowiedź: `<p>Hello, World!</p>`

GET /health

Health check do monitoringu / orkiestracji.

Odpowiedź:

```
{"status": "ok"}
```

GET /devices

Lista unikalnych `device_id` (alfabetycznie).

Zapytanie SQL:

```
SELECT DISTINCT device_id FROM measurements ORDER BY device_id
```

Odpowiedź:

```
["esp32-A1B2C3D4E5F6", "esp32-F88DAB004F8C"]
```

Pusta lista `[]` jeśli baza pusta.

GET /latest

Ostatni pomiar dla każdej pary `(device_id, sensor)`. Wykorzystuje PostgreSQL-specific `DISTINCT ON`.

Parametry:

- `device_id` (opcjonalny) — ogranicza wynik do jednego urządzenia.

Przykłady:

```
curl http://localhost:5001/latest  
curl "http://localhost:5001/latest?device_id=esp32-F88DAB004F8C"
```

Zapytanie SQL (bez filtra):

```
SELECT DISTINCT ON (device_id, sensor)  
    device_id, sensor, value, unit, ts_ms, received_at  
FROM measurements  
ORDER BY device_id, sensor, received_at DESC
```

Odpowiedź:

```
[
  {
    "device_id": "esp32-F88DAB004F8C",
    "sensor": "pressure",
    "value": 1013.25,
    "unit": "hPa",
    "ts_ms": 1742030400000,
    "received_at": "2026-05-17T19:30:15.123456"
  },
  {
    "device_id": "esp32-F88DAB004F8C",
    "sensor": "temperature",
    "value": 24.5,
    "unit": "C",
    "ts_ms": 1742030400000,
    "received_at": "2026-05-17T19:30:15.123456"
  }
]
```

GET /latest/temperature

Ostatnia temperatura dla każdego urządzenia (tylko `sensor = 'temperature'`).

Parametry:

- `device_id` (opcjonalny).

Przykład:

```
curl http://localhost:5001/latest/temperature
```

Odpowiedź: jak `/latest` , ale tylko temperatury.

GET /latest/pressure

Ostatnie ciśnienie dla każdego urządzenia (tylko `sensor = 'pressure'`).

Parametry:

- `device_id` (opcjonalny).

Przykłady:

```
curl http://localhost:5001/latest/pressure
curl "http://localhost:5001/latest/pressure?device_id=esp32-F88DAB004F8C"
```

Odpowiedź: jak `/latest` , ale tylko ciśnienia.

GET /history

Historia pomiarów z filtrowaniem i paginacją.

Parametry:

- `device_id` (opcjonalny) — filtruje po urządzeniu.
- `sensor` (opcjonalny) — filtruje po typie sensora.

- `limit` (opcjonalny, domyślnie `50`) — maks. liczba rekordów.

Przykłady:

```
curl "http://localhost:5001/history?limit=10"
curl "http://localhost:5001/history?device_id=esp32-F88DAB004F8C&sensor=temperature&limit=100"
```

Zapytanie SQL (z wszystkimi filtrami):

```
SELECT device_id, sensor, value, unit, ts_ms, received_at
FROM measurements
WHERE TRUE
      AND device_id = %s
      AND sensor = %s
ORDER BY received_at DESC
LIMIT %s
```

Odpowiedź: lista rekordów (DESC po `received_at`):

```
[
  {
    "device_id": "esp32-F88DAB004F8C",
    "sensor": "temperature",
    "value": 24.7,
    "unit": "C",
    "ts_ms": 1742030410000,
    "received_at": "2026-05-17 19:30:25.123456"
  },
  {
    "device_id": "esp32-F88DAB004F8C",
    "sensor": "temperature",
    "value": 24.6,
    "unit": "C",
    "ts_ms": 1742030405000,
    "received_at": "2026-05-17 19:30:20.234567"
  }
]
```

Uwaga — `received_at` w `/history` jest zwracane przez `str(...)`, a w `/latest` przez `.isoformat()`. Format minimalnie różny (`" "` vs `"T"`). Klient powinien akceptować oba.

Test w przeglądarce

Wszystkie endpointy są GET, więc otwierasz w przeglądarce:

- <http://localhost:5001/health>
- <http://localhost:5001/devices>
- <http://localhost:5001/latest>
- <http://localhost:5001/latest/temperature>
- <http://localhost:5001/history?limit=10>

Kody HTTP

Aktualna implementacja **zawsze zwraca 200** (nawet gdy lista pusta). Nie ma `404` dla nieznanego urządzenia ani `400` dla błędnych parametrów.

Jeśli serwer się wywróci (np. baza niedostępna) — Flask zwróci 500 z HTML-em debugowym (bo `debug=True`).

Bezpieczeństwo

- **Uwierzytelnianie (HTTP Basic Auth, lab 11)** — wszystkie endpointy zwracające dane są chronione; publiczne pozostają tylko `/` i `/health`. Szczegóły w sekcji „Uwierzytelnianie API — HTTP Basic Auth”.
- **CORS niezdefiniowany** — przeglądarka z innego origin (np. `file://`) zablokuje. Do dodania: `flask-cors` jeśli klient webowy.
- **SQL Injection** — zabezpieczony przez parametryzację `%s` w `psycopg2` (nie ma sklejanego stringów).
- `debug=True` — w produkcji wyłączyć (ujawnia stack trace + pozwala na zdalny exec przez Werkzeug debugger).

Co warto dodać (znane luki)

- `/devices/<device_id>/sensors` — lista sensorów per urządzenie.
- **Zakres czasowy** — parametr `from` / `to` w `/history` (epoch ms lub ISO 8601).
- **Paginacja właściwa** — `offset` + `limit` lub `cursor`.
- **Agregacje** — np. `/history/aggregated?window=1m` (średnie na okno).
- **Obsługa błędów** — własne handlers 404 / 400 / 500 zwracające JSON zamiast HTML-a.
- **Strukturalne logowanie** — obecnie tylko `stdout` Flask.
- **Healthcheck głębszy** — `/health` powinien sprawdzać też połączenie z bazą (`SELECT 1`), nie tylko że aplikacja żyje.
- **Modularyzacja** — `models.py` jest pusty (`# TODO`), całe API to jeden plik `app.py`. Przy rozroście rozsądnie rozbić na blueprints.

8. Uwierzytelnianie API — HTTP Basic Auth

Warstwa Flask REST API jest zabezpieczona mechanizmem **HTTP Basic Authentication**. Endpoint diagnostyczny `/health` pozostaje publiczny; wszystkie endpointy zwracające dane wymagają poprawnego loginu i hasła.

Chronione endpointy

Endpoint	Dostęp	Uwagi
GET <code>/health</code>	publiczny	healthcheck
GET <code>/</code>	publiczny	"Hello World", brak danych
GET <code>/devices</code>	chroniony	lista urządzeń
GET <code>/latest</code>	chroniony	najnowsze odczyty (filtr świeżości)
GET <code>/latest/temperature</code>	chroniony	najnowsza temperatura
GET <code>/latest/pressure</code>	chroniony	najnowsze ciśnienie
GET <code>/history</code>	chroniony	historia pomiarów

Brak nagłówka `Authorization` lub błędne dane → `401 Unauthorized` + nagłówek `WWW-Authenticate: Basic realm="RSP API"`.

Konfiguracja loginu i hasła

Dane logowania pochodzą ze **zmiennych środowiskowych** (nie z kodu). Plik `.env` jest w `.gitignore`:

```
API_USERNAME=student
API_PASSWORD=student
```

Usługa `flask` w `docker-compose.yml` wczytuje je przez `env_file: .env`. Zmiana hasła: edytuj `.env`, a następnie `docker compose up -d --build flask`.

Implementacja: `api/auth.py` — dekorator `@auth_required` nałożony pod `@app.route(...)`; porównanie poświadczeń przez `secrets.compare_digest` (odporność na timing attack). Hasło nie jest wypisywane w logach.

Testy curl

```
# publiczny – 200 bez logowania
curl -i http://localhost:5001/health

# chroniony bez danych – 401 + WWW-Authenticate
curl -i http://localhost:5001/devices

# chroniony z błędnym hasłem – 401
curl -i -u student:zle http://localhost:5001/devices

# chroniony z poprawnym hasłem – 200 + JSON
curl -i -u student:student http://localhost:5001/devices

# historia z Basic Auth (Windows: adres w cudzysłowie)
curl -i -u student:student "http://localhost:5001/history?device_id=esp32-EC0EAD004F8C&sensor=temperature&limit=5"
```

Klient (dashboard Streamlit) — odpowiednik „Części D / LabVIEW”

W tym projekcie warstwą prezentacji jest dashboard **Streamlit** (`wykresy_python/`), a nie LabVIEW (zarchiwizowany w `ui/`). Część D instrukcji zrealizowano w dashboardzie:

- panel boczny: checkbox **Użyj Basic Auth**, pola **Login** i **Hasło** (hasło maskowane, `type="password"`),
- klient `api_client.py` wysyła `Authorization: Basic ...` (parametr `auth=` w `requests`),
- obsługa `401`: czytelny komunikat „błędny login lub hasło”, bez nadpisywania wykresu pustymi danymi; w panelu bocznym wskaźnik statusu autoryzacji (`Autoryzacja: OK ✓ / 401`).

Uruchomienie dashboardu:

```
streamlit run wykresy_python/app.py
```

Ograniczenia Basic Auth

- login i hasło są wysyłane przy **każdym** żądaniu; Base64 to kodowanie, nie szyfrowanie,
- mechanizm jest bezpieczny tylko po **TLS/HTTPS** — bez tego dane logowania można podsłuchać,

- brak ról, wygasania sesji i rotacji haseł — wymagałyby osobnej logiki,
- kierunek docelowy: tokeny Bearer/JWT, OAuth2/OIDC, mTLS albo API Gateway (poza zakresem lab).

9. Dashboard webowy (Streamlit)

Warstwa prezentacji danych w przeglądarce — katalog `wykresy_python/`. Konsumuje REST API (Flask) i pokazuje aktualne pomiary, wykresy trendu oraz historię.

Zastępuje LabVIEW jako główną warstwę prezentacji. LabVIEW pozostaje w repozytorium jako archiwum — patrz `labview.md`.

Dlaczego Streamlit zamiast LabVIEW

- lekki dashboard w przeglądarce, bez licencji i ciężkiego IDE,
- ten sam kontrakt co LabVIEW: czyta **wyłącznie przez REST API** (`http://localhost:5001`), nie zna schematu bazy,
- auto-odświeżanie, filtr dat, eksport CSV „z pudełka”,
- łatwo uruchomić na dowolnym komputerze (`pip install + streamlit run`).

Struktura

```
wykresy_python/  
├─ app.py           # aplikacja Streamlit (UI, wykresy, tabela)  
├─ api_client.py    # klient REST API (APIClient + APIError)  
├─ config.py        # ustawienia domyślne (URL, limity, timeouty)  
└─ requirements.txt # streamlit, pandas, plotly, requests, streamlit-autorefresh
```

Komponenty

`config.py` — ustawienia domyślne

Stała	Wartość	Znaczenie
<code>DEFAULT_BASE_URL</code>	<code>http://localhost:5001</code>	adres backendu (API)
<code>DEFAULT_HISTORY_LIMIT</code>	50	domyślny limit historii
<code>DEFAULT_REFRESH_INTERVAL</code>	10 s	interwał auto-odświeżania
<code>REQUEST_TIMEOUT</code>	5 s	timeout żądań HTTP

`api_client.py` — `APIClient`

Owija REST API w metody: `health()`, `devices()`, `latest()`, `latest_temperature()`, `latest_pressure()`, `history(device_id, sensor, limit)`. Korzysta z `requests.Session`. Błędy sieci (brak połączenia, timeout, HTTP) zamienia na wyjątek `APIError` z czytelnym komunikatem — dzięki temu UI pokazuje sensowny błąd zamiast crashować.

`app.py` — aplikacja Streamlit

Pasek boczny + trzy sekcje:

- **Pasek boczny** (`sidebar`): pole Base URL backendu, wskaźnik **online/offline** (z `/health`), suwak interwału odświeżania (5–60 s), suwak limitu historii (10–500), wybór urządzenia (`wszystkie` / konkretne), **zakres dat** (domyślnie ostatnia doba), przycisk „Odśwież teraz”.
- **Aktualne pomiary** (`section_current`): kafelki `st.metric` z ostatnią temperaturą i ciśnieniem per urządzenie.
- **Wykresy trendu** (`section_charts`): dwa wykresy Plotly (temperatura, ciśnienie) — linie **spline** z markerami, oś czasu, oś Y ze **stałym krokiem** (`dtick` = 0.5 °C / 1.0 hPa) i dopasowanym zakresem, z filtrem po dacie.
- **Historia pomiarów** (`section_history`): tabela `st.dataframe` + przycisk **eksportu CSV**.

Całość odświeża się automatycznie co wybrany interwał (`st.autorefresh`).

Uruchomienie

Dashboard działa **poza Dockerem** — jest klientem REST API i nie wchodzi do sieci kontenerów. Wystarczy dostęp do portu `5001`.

Wymagania

- Python 3.10 lub nowszy (`python --version`)
- Działający backend: `docker compose up -d` w głównym katalogu repo

Krok po kroku

1. Uruchom backend (jeśli jeszcze nie działa)

```
# w głównym katalogu repo
docker compose up -d
```

Sprawdź, czy API odpowiada:

```
curl http://localhost:5001/health
# oczekiwana odpowiedź: {"status": "ok"}
```

2. Przejdź do katalogu dashboardu

```
cd wykresy_python
```

3. (Zalecane) Utwórz wirtualne środowisko

```
# Windows
python -m venv .env
.venv\Scripts\activate

# Linux / macOS
python3 -m venv .env
source .venv/bin/activate
```

4. Zainstaluj zależności (tylko przy pierwszym uruchomieniu)

```
pip install -r requirements.txt
```

5. Uruchom dashboard

```
python -m streamlit run app.py
```

Na niektórych systemach komenda `streamlit` może nie być w PATH — `python -m streamlit` działa zawsze.

Przeglądarka otworzy się automatycznie pod adresem `http://localhost:8501`. W pasku bocznym powinien być widoczny komunikat **Backend: online** ✓.

Zmiana adresu backendu

Jeśli API działa pod innym adresem (np. zdalny serwer), wpisz go w polu **Base URL backendu** w pasku bocznym — bez restartu aplikacji.

Wyłączenie dashboardu

W terminalu, w którym działa Streamlit: `Ctrl+C`.

Typowe problemy

Objaw	Przyczyna	Rozwiązanie
Backend: offline X	Docker nie działa	<code>docker compose up -d</code>
<code>ModuleNotFoundError</code>	Brak zależności	<code>pip install -r requirements.txt</code>
Port 8501 zajęty	Inna instancja Streamlit	<code>streamlit run app.py --server.port 8502</code>
Brak danych na wykresach	ESP32 nie nadaje / baza pusta	Sprawdź <code>docker logs ingestor</code>

Mapowanie widoków na endpointy API

Widok	Endpoint
status backendu (online/offline)	<code>GET /health</code>
lista urządzeń (dropdown)	<code>GET /devices</code>
aktualne pomiary (kafelki)	<code>GET /latest/temperature</code> , <code>GET /latest/pressure</code>
wykresy trendu	<code>GET /history?sensor=...&limit=...</code>
historia + eksport CSV	<code>GET /history?limit=...</code>

Powiązania z innymi dokumentami

- `api.md` — endpointy REST konsumowane przez dashboard.
- `architektura.md` — miejsce warstwy prezentacji w przepływie.
- `labview.md` — poprzednia warstwa prezentacji (archiwum).

10. Niezawodność ESP32 — reconnect, status, LWT

Opis mechanizmów zwiększających niezawodność firmware ESP32 dodanych w ramach laboratorium 9: reconnect Wi-Fi, reconnect MQTT, topic statusowy, Last Will and Testament.

Cel

W poprzedniej wersji firmware funkcje `connectWiFi()` i `connectMQTT()` zawierały blokujące pętle `while`, które zatrzymywały `loop()` na czas nawiązywania połączenia. Skutki:

- Utrata Wi-Fi w runtime → urządzenie zawisa w `connectWiFi()` bez feedbacku.
- Utrata MQTT → `delay(2000)` w pętli reconnect blokował publikację.
- Brak informacji o stanie urządzenia z punktu widzenia brokera — gdy ESP zniknie (reset, odłączone zasilanie), broker nie wie o tym.

Nowa wersja jest **non-blocking**: `loop()` wykonuje wszystkie sprawdzenia i próby reconnect przez harmonogram na bazie `millis()`, bez blokujących oczekiwań.

Topic statusowy

```
lab/<group_id>/<device_id>/status
```

Przykład: `lab/g01/esp32-F88DAB004F8C/status`

Topic jest **techniczny**, oddzielony od topiców pomiarowych (`temperature`, `pressure`). Dzięki rozdzieleniu warstwa pomiarowa i diagnostyczna nie mieszają się — łatwiej filtrować w MQTT Explorer i łatwiej dodać po stronie backendu osobnego subskrybenta dla statusu.

Payload `online`

Publikowany z flagą **retained** natychmiast po udanym `mqttClient.connect()`:

```
{
  "device_id": "esp32-F88DAB004F8C",
  "status": "online",
  "ts_ms": 1742030400000
}
```

Retained ⇒ nowy subskrybent (MQTT Explorer, kolejny backend) od razu widzi ostatni znany stan urządzenia bez czekania na kolejny komunikat.

Payload `offline` (Last Will)

Deklarowany przez klienta podczas `mqttClient.connect()` i publikowany **przez brokera** w imieniu klienta wtedy, gdy klient zniknie niepoprawnie (reset, odłączone zasilanie, zerwana sesja TCP):

```
{
  "device_id": "esp32-F88DAB004F8C",
  "status": "offline"
}
```

Brak `ts_ms` — broker publikuje LWT po wykryciu rozłączenia, więc znacznik czasu z momentu connect byłby mylący.

LWT również retained $\Rightarrow$ nowy subskrybent zobaczy `offline`, jeśli urządzenie jest aktualnie odłączone.

Mechanizm reconnect Wi-Fi

```
const unsigned long WIFI_RETRY_MS = 5000;
unsigned long lastWifiAttemptMs = 0;

void connectWifiIfNeeded() {
    if (WiFi.status() == WL_CONNECTED) return;
    if (millis() - lastWifiAttemptMs < WIFI_RETRY_MS) return;
    lastWifiAttemptMs = millis();

    WiFi.disconnect();
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}
```

- Sprawdza `WiFi.status()` w każdej iteracji `loop()`.
- Ponowna próba `WiFi.begin(...)` **maks. raz na 5 s** (chroni przed spamowaniem stack-a sieciowego).
- Brak blokującego `while` — `loop()` natychmiast wraca i może obsłużyć inne rzeczy (np. ponowny test stanu MQTT w następnej iteracji).

Mechanizm reconnect MQTT

```
const unsigned long MQTT_RETRY_MS = 3000;
unsigned long lastMqttAttemptMs = 0;

bool connectMqttIfNeeded() {
    if (WiFi.status() != WL_CONNECTED) return false;
    if (mqttClient.connected()) return true;
    if (millis() - lastMqttAttemptMs < MQTT_RETRY_MS) return false;
    lastMqttAttemptMs = millis();

    String willPayload =
        "{\"device_id\":\"" + deviceId + "\",\"status\":\"offline\"}";

    bool ok = mqttClient.connect(
        deviceId.c_str(),
        topicStatus.c_str(), // willTopic
        0, // willQos
        true, // willRetain
        willPayload.c_str() // willMessage
    );

    if (ok) {
        publishStatus("online");
    }
    return ok;
}
```

- Pre-warunek: Wi-Fi UP. Bez sieci nie ma sensu próbować łączyć MQTT.
- Retry maks. raz na 3 s.
- `connect()` z LWT — broker zarejestruje wiadomość offline.
- Po sukcesie natychmiastowa publikacja statusu `online`.

Pętla główna

```
void loop() {
    connectWiFiIfNeeded();
    connectMqttIfNeeded();
    mqttClient.loop();           // keepalive klienta MQTT

    // (runtime retry BMP280 jeżeli niedostępny)

    if (millis() - lastMeasurementMs >= MEASUREMENT_PERIOD_MS) {
        lastMeasurementMs = millis();
        publishMeasurement();
    }

    delay(10); // oddaj czas stackom sieciowym
}
```

`delay(10)` to nie blokada logiki — to standardowa pauza FreeRTOS dla oddania czasu CPU stackom Wi-Fi/MQTT. Cała logika reconnect i publikacja nadal działa bez czekania na cokolwiek.

Scenariusze testowe

Test 1: Utrata Wi-Fi w runtime

Kroki:

1. ESP uruchomione, publikuje pomiary, status `online` widoczny w MQTT Explorer.
2. Wyłącz Wi-Fi w punkcie dostępowym (lub przenieś ESP poza zasięg).
3. Obserwuj UART (115200 baud).

Oczekiwane:

- Po chwili na UART: `[WiFi] Brak połączenia - probuje reconnect... co 5 s.`
- Brak nowych pomiarów na topicu `temperature / pressure`.
- Broker wykrywa rozłączenie i publikuje LWT `offline` na `lab/.../status` (czas wykrycia zależy od keepalive MQTT, domyślnie ~15 s w PubSubClient).
- Po przywróceniu Wi-Fi: log `[WiFi] OK`, potem `[MQTT] Probuje polaczyc... OK`, potem `[STATUS] {"device_id":..., "status":"online", ...}`.
- Pomiary wracają.

Test 2: Niedostępny broker MQTT

Kroki:

1. ESP uruchomione, publikacja działa.
2. Zatrzymaj kontener brokera: `docker stop broker`.
3. Obserwuj UART.

Oczekiwane:

- Na UART: `[MQTT] Probuje polaczyc... blad, rc=-2 co 3 s (rc=-2 = connection failed).`
- Wi-Fi nadal UP (urządzenie nie reagaby się rozłączać tylko dlatego, że broker padł).
- Brak nowych pomiarów (publikacja sprawdza `mqttClient.connected()`).
- Po `docker start broker`: udane reconnect, status `online`, pomiary wracają.

Test 3: Last Will (nieczyste rozłączenie)

Kroki:

1. W MQTT Explorer subskrybuj `lab/+/+/status` (lub konkretny topic).
2. ESP połączone, widoczny `online` (retained).
3. Odłącz zasilanie ESP **bez** czystego `disconnect`.
4. Czekaj na keepalive MQTT (~15 s przy domyślnych ustawieniach PubSubClient).

Oczekiwane:

- Po wygaśnięciu keepalive na topicu statusowym pojawia się `{"device_id": "...", "status": "offline"}` opublikowany przez brokera.
- Komunikat jest retained — pozostaje na topicu do czasu kolejnego `online` lub ręcznego wyczyszczenia.

Test 4: Powrót do publikacji po awarii

Kroki:

1. Wykonaj Test 1 lub Test 2.
2. Przywróć warunki pracy (Wi-Fi / broker).
3. Sprawdź w MQTT Explorer topic `temperature`, `pressure`, `status`.

Oczekiwane:

- Status: `online`.
- Pomiary lecą co 5 s.
- `seq` w pomiarach kontynuuje (nie resetuje się do 0 — bo nie było restartu ESP).

Co dalej (opcjonalne rozszerzenia z instrukcji)

- **Status** `reconnecting` — publikowany przed kolejną próbą reconnect MQTT, daje precyzyjniejszy obraz stanu.
- **Rozróżnienie błędów** — payload statusowy z polem `reason` (`wifi_down`, `mqtt_down`, `broker_unreachable`).
- **Backoff** — rosnące opóźnienia między próbami (5s → 10s → 20s...) zamiast stałych 5s/3s, redukuje obciążenie sieci przy długiej awarii.
- **Licznik nieudanych prób** — dodać do payloadu, można obserwować stabilność z poziomu UI.
- **Pole** `status` **w bazie / API** — backend rejestruje historię stanów (osobna tabela `device_status`), API endpoint `/devices/status`, dashboard pokazuje status każdego urządzenia.

Powiązania z innymi dokumentami

- `esp32.md` — ogólny opis firmware i hardware.
 - `message_contract.md` — kontrakt pomiarów MQTT (status to topic techniczny, ma uproszczony payload).
 - `architektura.md` — miejsce ESP32 w przepływie.
-

11. Bezpieczeństwo — TLS na MQTT

Opis wdrożenia szyfrowanej i uwierzytelnionej komunikacji MQTT (TLS) między ESP32 a brokerem Mosquitto, wraz z izolacją usług w Dockerze.

Cel

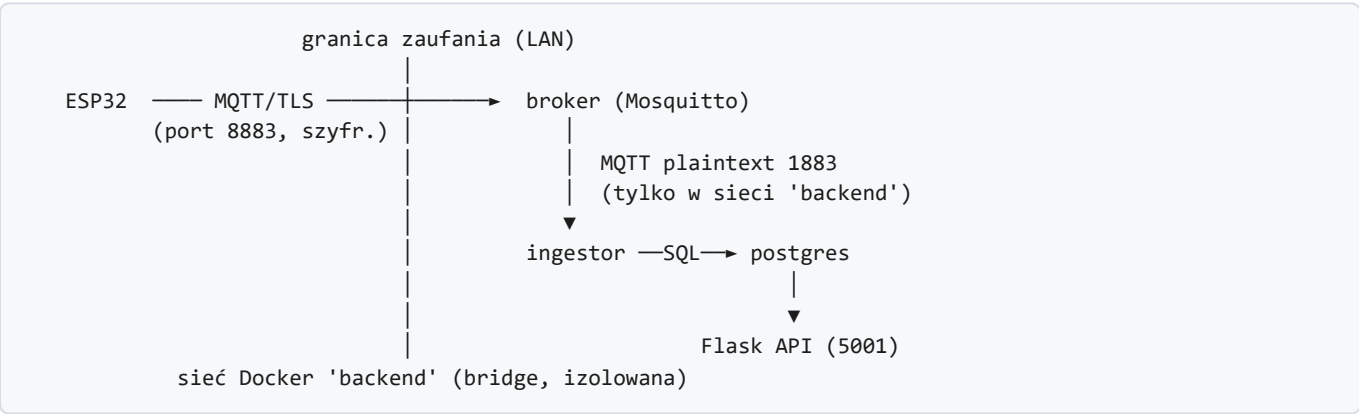
Punktem wyjścia był broker MQTT bez zabezpieczeń: nasłuch na porcie 1883, ruch w postaci jawnej (plaintext), `allow_anonymous true`, brak weryfikacji tożsamości stron. W takiej konfiguracji:

- dane pomiarowe można podsłuchać (sniffing w sieci Wi-Fi),
- można je zmodyfikować w locie (brak integralności),
- klient nie ma pewności, że łączy się z właściwym brokerem (ryzyko man-in-the-middle).

TLS (Transport Layer Security) usuwa te problemy: szyfruje kanał, gwarantuje integralność i — dzięki certyfikatowi CA — pozwala klientowi **zweryfikować tożsamość brokera**.

Model bezpieczeństwa — TLS na granicy zaufania

Zamiast wymuszać TLS na całej komunikacji, szyfrowanie zastosowano **na granicy zaufania**: tam, gdzie ruch realnie opuszcza maszynę i wchodzi do sieci Wi-Fi.



Połączenie	Port	Szyfrowanie	Uzasadnienie
ESP32 → broker	8883	TLS	ruch idzie przez sieć Wi-Fi LAN — realne ryzyko podsłuchu
ingestor → broker	1883	brak	ruch nie opuszcza izolowanej sieci kontenerów Docker
api → postgres	5432	brak	jw. — wewnątrz sieci backend, port nieeksponowany na hosta

Dzięki temu ingestor (klient w Pythonie) działa bez zmian, a mimo to cała komunikacja przekraczająca granicę maszyny jest zaszyfrowana. Rozwiązanie jest spójne z izolacją usług opisaną niżej.

Rozważona alternatywa — pełny TLS (broker nasłuchuje tylko na 8883, ingestor też przez TLS): bliższa dosłownej treści instrukcji, ale wymaga przerobienia ingestora i wprowadza więcej punktów awarii. Pozostawiona jako możliwy kierunek rozwoju (patrz [Ograniczenia i co dalej](#)).

Własny urząd certyfikacji (CA) i certyfikat serwera

W projekcie użyto **własnego CA** do podpisania certyfikatu brokera. Daje to pełną kontrolę nad zaufaniem i symuluje infrastrukturę PKI bez zewnętrznego dostawcy certyfikatów.

Wygenerowane pliki (`certs/`)

Plik	Rola	Tajny?
<code>ca.key</code>	klucz prywatny CA	tak — nigdy nie udostępniać
<code>ca.crt</code>	certyfikat CA (publiczny)	nie — trafia do klientów
<code>server.key</code>	klucz prywatny brokera	tak
<code>server.csr</code>	żądanie podpisania certyfikatu	—
<code>server.crt</code>	certyfikat brokera podpisany przez CA	nie
<code>openssl.cnf</code>	konfiguracja (subject + SAN)	nie

Katalog `certs/` jest w `.gitignore` — **klucze prywatne nie trafiają do repozytorium**. Publiczny `ca.crt` jest dodatkowo wkompiłowany w firmware ESP32.

Komendy generujące

Najszybciej — skrypt. Z katalogu projektu uruchom `bash generuj-certy.sh <IP_HOSTA>` (np. `bash generuj-certy.sh 156.17.45.169`). Skrypt utworzy cały katalog `certs/` — `openssl.cnf` z poprawnym SAN, CA oraz certyfikat serwera dla podanego IP. CA jest reużywane przy ponownym uruchomieniu (zmiana IP nie wymaga ponownego wgrywania firmware ESP32). IP hosta sprawdzisz przez `ipconfig` (Windows → Wi-Fi → IPv4) lub `hostname -I` (Linux/WSL). Poniżej ta sama procedura wykonana ręcznie, krok po kroku:

Uwaga (Windows / Git Bash): leading `/` w `-subj` bywa zamieniany na ścieżkę Windows. Jeśli `openssl req` zgłasza błąd subjectu, ustaw `export MSYS_NO_PATHCONV=1` przed komendami.

```
cd certs

# 1. CA – klucz + samopodpisany certyfikat (10 lat)
openssl genrsa -out ca.key 2048
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt \
  -subj "/C=PL/O=PWr Lab RSP/CN=RSP Lab Root CA" \
  -config openssl.cnf -extensions v3_ca

# 2. Serwer (broker) – klucz + CSR
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr \
  -subj "/C=PL/O=PWr Lab RSP/CN=156.17.45.51" \
  -config openssl.cnf

# 3. Podpisanie certyfikatu serwera przez CA (z rozszerzeniem SAN)
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out server.crt -days 3650 -extfile openssl.cnf -extensions v3_req
```

Subject Alternative Name (SAN)

Certyfikat serwera obejmuje nazwy, pod którymi broker bywa adresowany:

```
DNS:localhost, DNS:broker, DNS:156.17.45.51, IP Address:127.0.0.1, IP Address:156.17.45.51
```

`156.17.45.51` to adres IP komputera z Dockerem w sieci Wi-Fi laboratorium — po nim łączy się ESP32. **IP jest wpisane zarówno jako IP, jak i jako DNS** — to celowe obejście ograniczenia mbedTLS na ESP32 (szczegóły

w sekcji [Napotkane problemy](#)).

Konfiguracja brokera (Mosquitto)

broker/mosquitto.conf — dwa listenery:

```
persistence true
persistence_location /mosquitto/data/
log_dest stdout

# Ustawione PRZED listenerami => globalne dla obu
allow_anonymous true

# Listener wewnętrzny (plaintext) – tylko dla usług w sieci Docker (ingestor)
listener 1883

# Listener TLS – dla ESP32 i klientów zewnętrznych
listener 8883
cafile /mosquitto/certs/ca.crt
certfile /mosquitto/certs/server.crt
keyfile /mosquitto/certs/server.key
```

Certyfikaty są **montowane** do kontenera (wolumen `:ro`), a nie kopiowane do obrazu — dzięki temu klucz prywatny serwera nie łąduje w warstwach obrazu Dockera. `require_certificate` pozostaje domyślnie `false`: klient weryfikuje brokera, ale broker nie wymaga certyfikatu klienta (uwierzytelnianie jednostronne).

Izolacja usług (docker-compose)

W ramach lab 10 wzmocniono też izolację środowiska Docker:

```
services:
  flask:
    ports: ["5001:5001"]      # API publiczne – jedyny port aplikacyjny na hoście
    networks: [backend]
  broker:
    ports: ["8883:8883"]      # tylko TLS wystawiony na hosta (1883 zostaje wewnątrz)
    volumes: ["/certs:/mosquitto/certs:ro"]
    networks: [backend]
  database:
    # brak mapowania portu 5432 – baza nieosiągalna z hosta
    networks: [backend]
  ingestor:
    networks: [backend]
networks:
  backend:
    driver: bridge
```

Efekty bezpieczeństwa:

- **baza danych nie jest dostępna z hosta** (usunięto `5432:5432`),
- **broker przyjmuje połączenia z zewnątrz wyłącznie po TLS** (`8883`); port `1883` istnieje tylko wewnątrz sieci `backend`,
- usługi komunikują się w dedykowanej sieci bridge, odseparowane od sieci zewnętrznej.

Klient ESP32 (TLS)

Zmiany w `esp32/src/main.cpp` (reszta logiki z lab 9 — reconnect, status, LWT — bez zmian; szyfrowany jest jedynie transport):

```
#include <WiFiClientSecure.h>

// Certyfikat CA (publiczny) – odpowiada certs/ca.crt
static const char* CA_CERT = R"EOF(
-----BEGIN CERTIFICATE-----
... (treść ca.crt) ...
-----END CERTIFICATE-----
)EOF";

WiFiClientSecure espClient;          // zamiast WiFiClient
PubSubClient mqttClient(espClient);

void setup() {
    // ...
    espClient.setCACert(CA_CERT);      // weryfikacja tożsamości brokera
    mqttClient.setServer(MQTT_HOST, MQTT_PORT); // MQTT_PORT = 8883 (secrets.h)
}
```

Zależność od zegara: TLS sprawdza daty ważności certyfikatu, więc ESP musi mieć ustawiony czas. Realizuje to `syncNTP()` w `setup()` (z lab wcześniejszych). Bez synchronizacji NTP weryfikacja certyfikatu może się nie powieść.

Testy i weryfikacja

Wszystkie testy wykonane na działającym systemie; wyniki rzeczywiste.

T1 — handshake TLS z weryfikacją CA (pozytywny)

```
openssl s_client -connect 156.17.45.51:8883 -CAfile certs/ca.crt -verify_ip 156.17.45.51
# subject=C=PL, O=PWr Lab RSP, CN=156.17.45.51
# issuer=C=PL, O=PWr Lab RSP, CN=RSP Lab Root CA
# Verify return code: 0 (ok)
```

T2 — handshake bez CA (kontrola negatywna)

```
openssl s_client -connect 156.17.45.51:8883      # bez -CAfile
# Verify return code: 19 (self-signed certificate in certificate chain)
```

Bez certyfikatu CA klient **nie ufa** certyfikatowi brokera — dowód, że weryfikacja faktycznie działa (nie akceptuje dowolnego certyfikatu).

T3 — publikacja po TLS

```
docker exec broker mosquitto_pub -h localhost -p 8883 \
  --cafile /mosquitto/certs/ca.crt -t test/tls -m "TLS działa poprawnie"
# exit 0 → publikacja po zaszyfrowanym kanale powiodła się
```

T4 — plaintext na porcie TLS (kontrola negatywna)

Próba połączenia bez TLS na port `8883` jest odrzucana przez brokera:

```
OpenSSL Error ... error:0A00010B:SSL routines::wrong version number
Client ... disconnected: Protocol error. (First packet not CONNECT)
```

Potwierdza, że listener `8883` wymusza TLS i odrzuca ruch jawny.

T5 — ESP32 łączy się po TLS

Log brokera po wgraniu firmware:

```
New client connected from 172.20.0.1 as esp32-EC0EAD004F8C (p4, c1, k15) on port 8883
```

Połączenie na **porcie 8883** = urządzenie używa TLS.

T6 — test end-to-end (ESP → TLS → broker → ingestor → baza → API)

```
curl http://localhost:5001/latest/temperature
# {"device_id":"esp32-EC0EAD004F8C","sensor":"temperature","value":24.9,"unit":"C", ...}
curl http://localhost:5001/latest/pressure
# {"device_id":"esp32-EC0EAD004F8C","sensor":"pressure","value":1015.25,"unit":"hPa", ...}
```

Świeże dane w API = cały łańcuch działa po wdrożeniu TLS.

Napotkane problemy i rozwiązania

mbedtls na ESP32 nie dopasowuje IP w SAN

Objaw: po pierwszym wgraniu firmware (`setCACert` + cert z SAN `IP:156.17.45.51`) ESP łączył się po TCP/TLS, ale handshake kończył się błędem:

```
(-9984) X509 - Certificate verification failed
```

mimo że z hosta `openssl s_client -verify_ip` weryfikował ten sam certyfikat poprawnie (`Verify return code: 0`).

Przyczyna: ESP łączy się po **adresie IP**. Biblioteka mbedtls (stos TLS na ESP32) porównuje string hosta podany do połączenia wyłącznie z wpisami SAN typu **DNS**, a nie z wpisami typu **IP**. OpenSSL (inna implementacja) potrafi dopasować IP-SAN, dlatego test z hosta przechodził, a ESP — nie.

Rozwiązanie: dopisanie adresu IP **również jako wpisu SAN typu DNS** (`DNS:156.17.45.51` obok `IP:156.17.45.51`). Wtedy porównanie po stringu przechodzi i pełna weryfikacja certyfikatu zostaje zachowana.

Co istotne — wymaga to jedynie **przepisania certyfikatu serwera** (tym samym CA) i restartu brokera. CA się nie zmienia, więc **firmware ESP32 nie wymaga ponownego wgrania**:

```
# po edycji certs/openssl.cnf (dodanie DNS.3 = <IP>):
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
-out server.crt -days 3650 -extfile openssl.cnf -extensions v3_req
docker compose restart broker
# weryfikacja jak po stronie mbedtls (dopasowanie po stringu/DNS):
openssl s_client -connect 156.17.45.51:8883 -CAfile certs/ca.crt -verify_hostname 156.17.45.51
# Verify return code: 0 (ok)
```

Odrzucone alternatywy:

- `espClient.setInsecure()` — wyłącza weryfikację (szyfrowanie bez uwierzytelnienia brokera). Sprzeczne z celem lab.
- Łączenie po nazwie DNS zamiast IP — w sieci laboratorium brak DNS rozwiązującego nazwę brokera.

Ograniczenia i co dalej

- **Self-signed CA** — zaufanie lokalne (nie globalny urząd certyfikacji). Dla systemu laboratoryjnego / przemysłowego zamkniętego jest to akceptowalne.
- **IP w certyfikacie** — zmiana adresu IP hosta wymaga przepisania `server.crt` (CA pozostaje). Patrz procedura wyżej.
- **Broker anonimowy** — uwierzytelnianie jest jednostronne (klient weryfikuje brokera, broker nie weryfikuje klienta). Naturalny następny krok:
 - **mutual TLS** — certyfikat klienta po stronie ESP32 (`require_certificate true`),
 - lub **login/hasło + ACL** (`password_file` , `acl_file`).
- **Plaintext 1883 wewnątrz sieci Docker** — do pełnego TLS należałoby przerobić również ingestor na połączenie szyfrowane.

Powiązania z innymi dokumentami

- architektura.md — miejsce TLS w przepływie danych.
- esp32.md — ogólny opis firmware.
- reliability_esp32.md — niezawodność (lab 9); logika reconnect/LWT działa pod TLS bez zmian.
- uruchomienie.md — start środowiska (port `8883`).

12. Dodatek: LabVIEW UI (archiwum)

⚠ **ARCHIWUM.** Zrezygnowano z LabVIEW jako warstwy prezentacji — zastąpił go dashboard webowy w Streamlit (wykresy.md). Katalog `ui/` i ten dokument zostają w repo jako archiwum/alternatywa.

Opis

Aplikacja desktopowa w LabVIEW pełniąca rolę warstwy prezentacji. Konsumuje REST API (Flask, `localhost:5001`), prezentuje aktualne pomiary i historię w postaci dashboardu. Działa **poza Dockerem** — to natywna aplikacja desktopowa łącząca się z backendem przez HTTP.

Miejsce w architekturze

```
PostgreSQL —SQL→ Flask REST API —HTTP/JSON→ LabVIEW UI
                        (localhost:5001)
```

LabVIEW NIE łączy się bezpośrednio z bazą — komunikacja idzie przez REST. Powód: stabilny kontrakt JSON, ukrycie schematu bazy, filtrowanie po stronie backendu (patrz `architektura.md`).

Wymagania

- **LabVIEW 2024 Q3** lub nowszy (pliki *.vi zapisane w 24.3.1, *.ctl w 24.1.1).
- **JKI REST Client** — biblioteka do obsługi HTTP/REST. Instalacja przez VI Package Manager (VIPM):

Tools → VI Package Manager → search "JKI REST Client" → Install

W VI używane są: Create REST Client.vi, HTTP GET.vi, Destroy REST Client.vi.

- **Działający backend** — uruchomione docker compose up -d, endpoint http://localhost:5001/health zwraca {"status":"ok"}.

Struktura katalogu

```
labview/  
├─ epoch to cluster.vi      # Konwersja Unix epoch (ms) → timestamp/cluster  
└─ template/  
    ├─ main.vi              # Główny VI z UI (front panel + block diagram)  
    ├─ kontrakt.ctl         # Typedef cluster odwzorowujący kontrakt API  
    └─ measure_data.ctl     # Typedef cluster pojedynczego pomiaru
```

epoch to cluster.vi

VI pomocniczy. Wejście: ts_ms z API (Unix epoch w **milisekundach**). Wyjście: cluster z rozbiem czasu (rok / miesiąc / dzień / godzina / minuta / sekunda) lub natywny LabVIEW timestamp — gotowy do wyświetlenia w UI / podpięcia pod oś czasu wykresu.

Powód osobnego VI: API zwraca czas jako liczbę w ts_ms (z urządzenia po NTP) oraz received_at (ISO 8601). LabVIEW pracuje natywnie na typie *Timestamp* — potrzebna konwersja.

template/measure_data.ctl

Typedef cluster pojedynczego pomiaru — odpowiednik jednego rekordu JSON z /latest lub /history. Pola odpowiadają polom z API:

Pole LabVIEW (cluster)	Pole JSON	Typ
device_id	device_id	String
sensor	sensor	String
value	value	Double
unit	unit	String
ts_ms	ts_ms	I64 (epoch ms)
received_at	received_at	String / Timestamp

Cluster jest *typedef* — zmiana definicji propaguje się do wszystkich VI które go używają.

template/kontrakt.ctl

Typedef cluster z **parametrami żądania** do API — agreguje filtry przekazywane do endpointów /latest, /latest/temperature, /history:

- `device_id` (opcjonalny) — filtr na konkretne urządzenie,
- `sensor` (opcjonalny) — filtr na typ sensora (`temperature` / `pressure`),
- `limit` (opcjonalny, domyślnie 50) — maks. liczba rekordów w `/history` .

Powód osobnego typedefu: jeden punkt zmiany kontraktu — gdy API zyska nowy parametr (np. `from` / `to`), edytuje się `kontrakt.ct1` i wszystkie miejsca w `main.vi` automatycznie podchwytyją pole.

`template/main.vi`

Główny VI z front panelem (UI) i diagramem blokowym (logika). Cykl:

1. **Inicjalizacja** — `Create REST Client.vi` (JKI) tworzy obiekt klienta REST z `base URL` = `http://localhost:5001` .
2. **Pobieranie danych** — `HTTP GET.vi` woła endpoint (np. `/latest` lub `/history`) z parametrami z `kontrakt.ct1` . Odpowiedź to JSON.
3. **Parsowanie JSON** — JSON → tablica klustrów `measure_data.ct1` (przez `Unflatten From JSON` z natywnej biblioteki LabVIEW lub funkcję JKI).
4. **Konwersja czasu** — `ts_ms` (I64) → timestamp przez `epoch to cluster.vi` .
5. **Prezentacja** — wartości na wskaźnikach, tablica historii w *Table*, wykres trendu (XY Graph).
6. **Sprzątanie** — `Destroy REST Client.vi` przy zamykaniu VI.

Uruchomienie

1. Backend musi działać:

```
docker compose up -d
curl http://localhost:5001/health      # {"status":"ok"}
curl http://localhost:5001/latest     # niepusta tablica
```

2. Otwórz `labview/template/main.vi` w LabVIEW.
3. Jeśli pierwsze otwarcie po klonie repo — LabVIEW może zapytać o zależności (JKI REST Client). Jeśli brak, doinstaluj przez VIPM.
4. Uruchom (białą strzałką *Run*). Po kilku sekundach widoczne dane.

Mapowanie endpointów na widoki

Widok / akcja	Endpoint API
Lista urządzeń (dropdown)	GET <code>/devices</code>
Aktualne pomiary (kafelki)	GET <code>/latest</code>
Aktualne pomiary danego urządzenia	GET <code>/latest?device_id=...</code>
Tylko temperatura (ostatnia)	GET <code>/latest/temperature</code>
Tylko ciśnienie (ostatnie)	GET <code>/latest/pressure</code>
Wykres trendu temperatury	GET <code>/history?sensor=temperature&limit=N</code>
Filtr device + sensor + limit	GET <code>/history?device_id=...&sensor=...&limit=...</code>

Format czasu — niuans

API zwraca `received_at` w dwóch wariantach:

- `/latest`: ISO 8601 z `T` jako separatorem (`2026-05-17T19:30:15.123456`),
- `/history`: format z separatorem spacji (`2026-05-17 19:30:15.123456`).

Parser w LabVIEW (`Scan From String`, format ISO 8601) powinien akceptować oba — lub używać `ts_ms` (l64) jako głównego źródła czasu i `received_at` tylko do wyświetlenia.

Co warto dodać

- **Healthcheck przed wywołaniem** — wywołanie `/health` przy starcie VI, jasny komunikat błędu gdy backend nie działa.
- **Auto-refresh** — pętla *Timed Loop* z konfigurowalną częstotliwością (np. co 5 s), zatrzymywalna przyciskiem.
- **Obsługa pustej bazy** — `/latest` zwraca `[]` gdy brak pomiarów; UI powinien to pokazać sensownie zamiast pustego wykresu.
- **Wykres historii dla wybranego urządzenia** — kombinacja dropdownu z `/devices` i `/history?device_id=...`.
- **Eksport CSV** — zapis tabeli historii do pliku.
- **Konfigurowalny base URL** — kontrolka stringowa zamiast hardcoded `localhost:5001` (do podpięcia pod inny host w sieci lokalnej).

13. Test end-to-end

Pełna ścieżka weryfikacji całego systemu po uruchomieniu:

1. Uruchom Compose: `docker compose up -d --build`.
2. Wgraj firmware na ESP32 (skonfigurowany `secrets.h`).
3. ESP32 publikuje co 5 s na dwa topiki (temperature, pressure).
4. Sprawdź logi ingestora: powinno lecieć `[OK] Zapisano...`.
5. Sprawdź bazę:

```
docker exec -it postgres psql -U admin -d abcd_db \  
-c "SELECT device_id, sensor, value, unit, received_at FROM measurements ORDER BY id DESC LIMIT 10;"
```

6. Sprawdź API (z danymi Basic Auth): `curl -u $API_USERNAME:$API_PASSWORD http://localhost:5001/latest` powinno zwrócić ostatnie pomiary.
7. Uruchom dashboard: `cd wykresy_python && python -m streamlit run app.py`. Pod `http://localhost:8501` powinny być widoczne aktualne kafelki i wykresy z danymi z ESP32.

14. Status realizacji (laboratoria)

Lab	Temat	Status
0	Architektura, narzędzia	OK
1	Onboarding, Docker, WSL	OK
2	ESP32 dummy + Wi-Fi	Pominięte (od razu BMP280)
3	ESP32 + MQTT publish	OK (<code>esp32/src/main.cpp</code>)
4	Kontrakt danych	OK (<code>docs/message_contract.md</code>)
5	Ingestor MQTT → DB	OK (<code>ingestor/</code>)
6	REST API	OK (<code>api/</code>)
7–8	Prezentacja danych	Streamlit (<code>wykresy_python/</code>); LabVIEW w archiwum
9	Niezawodność (reconnect, LWT, status)	OK (<code>esp32/src/main.cpp</code> , <code>docs/reliability_esp32.md</code>)
10	Security MQTT — TLS + izolacja usług	OK (<code>broker/</code> , <code>docs/security_tls.md</code>)
11	Bezpieczeństwo API — HTTP Basic Auth	OK (<code>api/auth.py</code> , <code>docs/basic_auth.md</code>)